

# Towards Latency-Aware Linux Scheduling for Serverless Workloads

Al Amjad Tawfiq Isstaif  
University of Cambridge  
Cambridge, United Kingdom  
alamjad.isstaif@cl.cam.ac.uk

Richard Mortier  
University of Cambridge  
Cambridge, United Kingdom  
richard.mortier@cl.cam.ac.uk

## Abstract

A key principle in the design of the Linux kernel’s Completely Fair Scheduler (CFS) is fairness: all running tasks receive a minimum time slice during every scheduling period, ensuring that none starve. However, this may lead to a significant number of context switches when a server is overloaded with a large number of colocated tasks, which may cause significant degradation in server performance. Unfortunately, this situation is exactly what we found when hosting serverless-style workloads which typically consist of a large number of short-lived, CPU-bound functions sharing resources. We propose modifying the Linux CFS to mitigate this problem by giving priority to the long tail of least loaded functions. These are the functions which are mostly idle and only run occasionally for a short while after being triggered unexpectedly. The large number of such functions in serverless environments means that prioritising them helps drain contended CPU run queues, reducing the total overhead due to context switching, thereby improving the performance not only of the prioritised functions but other functions as well. We implement this policy in the Linux kernel scheduler and demonstrate how it integrates well with Knative, an open source Kubernetes-based serverless framework. Given contention scenarios synthesised from real-world traces, our modified CFS can introduce a 5-30% increase in attainment of latency targets.

## CCS Concepts

• Computer systems organization → Cloud computing; • Software and its engineering → Scheduling.

## Keywords

serverless, Linux, containers, CPU scheduling, Multi-tenancy, Resource management

## ACM Reference Format:

Al Amjad Tawfiq Isstaif and Richard Mortier. 2023. Towards Latency-Aware Linux Scheduling for Serverless Workloads. In *The 1st Workshop on Serverless Systems, Applications and Methodologies (SESAME ’23)*, May 8, 2023, Rome, Italy. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3592533.3592807>



This work is licensed under a Creative Commons Attribution International 4.0 License. *SESAME ’23, May 8, 2023, Rome, Italy*  
© 2023 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-0185-6/23/05.  
<https://doi.org/10.1145/3592533.3592807>

## 1 Introduction

Serverless computing is becoming an increasingly popular deployment model which allows developers to focus on the application functionality while delegating the server resource management to a service provider. Developers provide their applications as deployment units known as functions which are then instantiated by the serverless platform on-demand whenever a request is received for such a function. As such, a key characteristic of serverless workloads is burstiness [18] where functions can generate a large number of requests over a short time and remain idle for a significant time. In this context, workload consolidation techniques become an important feature where a service provider would share server resources among multiple tenants as each tenant would only use a small fraction of such resources. In this paper, we focus on the challenges associated with workload consolidation when a serverless platform is expected to execute requests within a given latency target [16, 22, 32, 34, 36, 38–40].

As perhaps the most popular underlying OS kernel for serverless deployments, this paper examines the CPU contention issues in the context of Linux’s Completely Fair Scheduler (CFS). This is the default scheduling policy for Linux and is widely used as an integral part of compute resource management of most container-based serverless systems [23, 27] as well as those based on lightweight VMs [15, 28]. The work-conserving design principle of CFS [26, 30] means it maximises the use and sharing of available CPU cores among bursty tasks in multi-core servers. However, its other design principle of “fair scheduling” leads to high context switch overheads causing increased turnaround times, which has a serious impact on latency under such high-contention workloads. This arises because CFS attempts to allocate a minimum time slice for every runnable task during every scheduling period. Given the very large number of tasks typical in a high-density serverless deployment (i.e. 100s-1000s tasks on a single server [28]), our experiments show that this can cause a significant performance degradation potentially causing latency targets to be missed for over 50% of requests under high CPU load (§3.2). This is inline with results reported by previous works [19, 28, 33] which confirm significant impact on the performance of colocated tasks due to the frequent context switches of CFS.

The Linux community is well aware of the latency issues that can arise under CFS and several solutions have been introduced to provide system administrators with more control over the latency of tasks that share the CPU cores of a host machine [10, 11, 14]. However, while useful when statically prioritising one group of tasks over another (e.g. interactive vs batch tasks), they do not address the problem of colocating multiple interactive tasks with latency-sensitive requirements. Google’s Autopilot [35] addresses

this issue by a vertical autoscaling solution that dynamically adjusts resource limits for colocated workloads. However, this solution assumes gradually changing demand of long-running services based on relatively large time windows (i.e. 5 minutes) which does not address the highly volatile and unexpected burst demand in serverless workloads. Another fundamental limitation of similar solutions based on dynamic resource partitioning [17, 29], is that these do not scale to large number of tasks expected in serverless deployments.

In this paper, we propose to relax the fairness goal of CFS in order to mitigate the impact of inevitable contention on the latency of serverless functions sharing the CPU resources of a single multi-core server. Instead, we propose a scheduling policy that gives priority to the long tail of least loaded functions found in highly skewed serverless workloads [24, 41], which also allows other functions to execute with significantly less interruption. The contributions are:

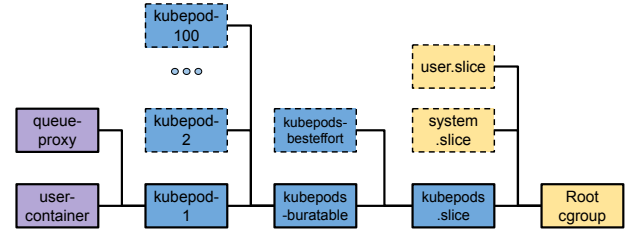
- (1) We evaluate the interaction between the CFS scheduler and the skewed demand distribution of realistic serverless workloads, quantifying the potential impact of contentions caused by CFS (§3).
- (2) We present a patch to the CFS scheduler that mitigates the impact of contention on the latency of serverless-style bursty tasks sharing the CPU resources of a Linux host (§4).
- (3) We evaluated our patch to CFS based on synthetic contention scenarios drawn from real-world traces and found that it offers 6-30% increase in attainment of large latency targets compared to CFS (§5).
- (4) Our proposed CFS prototype is highly responsive, requires no coordination with the Kubernetes control plane and can seamlessly integrate with Knative [2], an open source serverless framework (§6).

## 2 Task scheduling in Linux clusters

### 2.1 Linux scheduling in Knative

Knative is an open source serverless framework based on Kubernetes. We focus on Kubernetes-based serverless frameworks [27] due to the prevalence of Kubernetes as a container orchestration framework. A Kubernetes-based serverless framework such as Knative leverages containers as workers to execute user functions and depends on the Kubernetes API to manage the deployment and configuration of these containers. The interaction between Knative and the Linux scheduler happens through the Kubernetes compute resource abstractions which are in turn tightly coupled to the control knobs of the default Linux CFS scheduler. CFS exposes its control knobs via the Linux control groups interface [9] known as cgroups, which Kubernetes leverages to organise and configure its containers on each cluster node. Deploying a number of Knative functions on a cluster node results with the cgroup structure depicted in Figure 1.

Knative implements the typical sidecar pattern in which a user function is deployed as a user-container and proxied by a queue-proxy. The sidecar approach helps provide generic observability and security features for functions out of the box. The two containers are deployed as a single Kubernetes pod which is the smallest deployment unit in Kubernetes. In this work we focus on the scheduling performance given equal CPU shares for Kubernetes pods (via



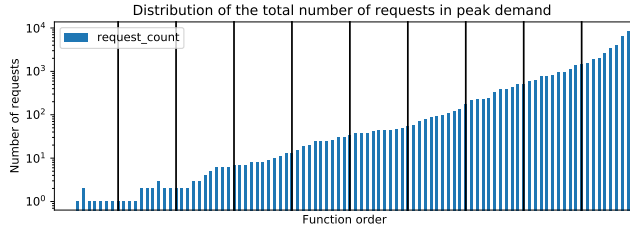
**Figure 1: Illustration of the cgroup hierarchy that is instantiated for 100 Knative functions on a Linux host in a Kubernetes cluster. The yellow entries are available by default in Linux, while the blue and purple entries correspond to Kubernetes and Knative respectively. Solid boxes illustrate entries which have all their child entries expanded.**

CPU requests and the underlying `cpu.shares` cgroup property). Given the dynamic and bursty nature of serverless functions and given the large number of these functions configuring all functions with an equal CPU share represents a sensible practical baseline. We also avoid the usage of the Kubernetes limits property and the underlying CFS bandwidth control which has been reported to be a problematic feature by Kubernetes practitioners [5] due to unnecessary throttling for containers which can cause latency problems. This issue is also known to be exacerbated by bursty tasks [7].

### 2.2 Linux’s Completely Fair Scheduler (CFS)

CFS implements a round-robin form of scheduling in which all runnable tasks receive a timeshare during every scheduling period. CFS schedules tasks on the granularity of threads and uses a data structure known as the scheduling entity (se) to track scheduling-related metrics and implement its various heuristics. CFS achieves the fairness goal by tracking the runtime of every scheduling entity (`se->vruntime`) and prioritising entities with the lowest runtime ensuring no scheduling entity is left behind. When a task is preempted because it has completed its timeshare, its `vruntime` is updated before it is added back to the run queue. Given the dynamic priority based on the minimum `vruntime`, increasing this value for an entity effectively means penalising its priority, which can provide other tasks with the opportunity to “catch up”.

Realisation of this fair share is non-trivial due to the CFS group scheduling feature [8] in which tasks are organised into a hierarchy that is parallel to the Linux control groups tree. The aim of this feature is to allocate a timeshare for a group of tasks as a whole, ensuring the total time of these task does not exceed this timeshare. For example, all processes and threads spawned by a container belong to the cgroup that corresponds to this container. No matter how many tasks a container may have, the total time available for this container will remain the same. CFS uses a set of group scheduling entities (gse) which also has a `vruntime` attribute to track the total execution of tasks belonging to a cgroup on a particular CPU core. This attribute is then used by CFS to determine the order of scheduling entities in a tree of run queues (`cfs_rq`) that mirrors the cgroup structure and is connected to the top level run queues for every CPU core (Figure 6). Additionally, CFS uses a task group data structure (tg) to maintain the configurations of a cgroup (e.g.



**Figure 2: Total number of requests per trace segment representing peak demand in the two week traces of Azure Function Invocation Trace [41]. Black vertical lines illustrate classification of traces into ten demand bands which are used later in deriving contention scenarios depicted in Figure 3a.**

cpu.shares) as well as the aggregated load metric  $tg \rightarrow load\_avg$  which we reuse as an alternative priority in §4.

### 3 Contention under serverless workloads

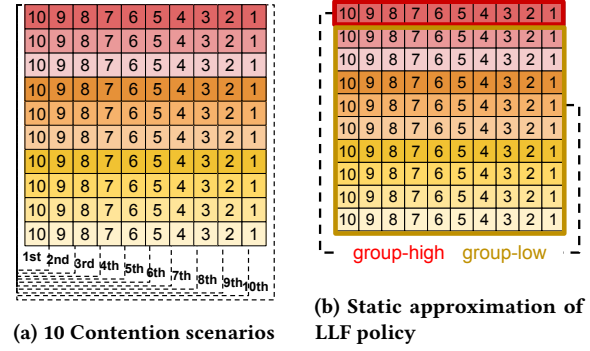
#### 3.1 A compute-bound serverless use case

In this section, we study the contention among a number of image classification functions which share the CPU resources of a single server in a clustered serverless environment. We focus on studying CPU contention in contexts dominated by stateless, compute-bound workloads where latency matters [1, 3, 6, 25, 37], e.g., image and video processing, cryptography and ML serving. We acknowledge that incorporating a mix of I/O-bound workloads [42] can potentially mitigate the impact of contention given the same number of functions. However, we focus on studying this issue under the pessimistic scenario given its high plausibility for compute-bound serverless use cases and its inevitability in a highly utilised serverless setup.

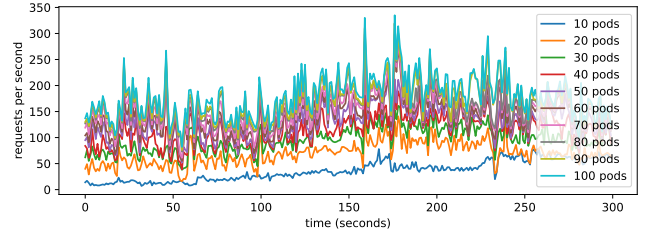
We synthesise contention scenarios based on the Azure Functions Invocation Trace [41]. The aim of these contention scenarios is to evaluate the impact of contention within the Linux scheduler on the latency of colocated functions under increasing CPU load levels. We focus on analysing the impact of contention during short 5 minute intervals in which the operating system scheduler plays a crucial role in comparison to other components in the cluster. Figure 2 illustrates the highly skewed distribution of demand generated by functions in the Azure Functions Invocation Trace [41]. The figure illustrates the peak demand that each of the 119 functions in the dataset can generate in a short time interval (5 minutes). Over the entire duration of the two week trace, we notice that the peak demand for most functions can be no more than tens or hundreds of requests.

We use the top 100 traces to develop a contention scenario in which a server is overloaded by 100 functions experiencing their peak demand. This corresponds to the maximum number of functions which can be realistically colocated in a Kubernetes cluster node given the hardcoded limit of 110 pods per node<sup>1</sup> (leaving a

<sup>1</sup>Considerations for large clusters <https://kubernetes.io/docs/setup/best-practices/cluster-large/>



**Figure 3: (a) methodology of deriving multiple contention scenarios by concurrently replaying multiple traces. Each cell in the grid corresponds to a function trace and darkest colours correspond to the most significant demand band. Numbers represent the order of each trace in its demand band given the number of its requests. (b) approximating a least loaded first scheduling policy by splitting functions into two groups.**



**Figure 4: Aggregated arrival rates for 10 contention scenarios derived based on the methodology depicted in Figure 3a.**

margin of 10 pods which is left for essential administrative purposes). We also study the impact of less intense contention with fewer numbers of functions before we reach the worst-case scenario. To ensure the representation of the full spectrum of demand levels in all contention scenarios, we classify the trace segments into 10 demand bands which are equally represented in every contention level. This approach enables us to evaluate the interaction of the CFS scheduler with the skewed demand distribution under increasing load levels. This results with 10 contention levels in which every level is derived by adding 10 function traces to the previous level. The exact methodology to compose these contention is depicted in Figure 3a which results with the aggregate arrival rates in Figure 4.

#### 3.2 Impact of contention on latency

Figure 5a illustrates the impact of increasing the level of contention on the percentage of requests which meet a set of multiple latency targets. We consider the total number of requests across all colocated functions starting from the lowest contention level and up to the worst-case contention scenario. For each contention level, we also plot the aggregated CPU utilisation over the duration of the experiment. We pick multiple latency targets values starting from the standard datacenter latency target of 1000ms [21], as well

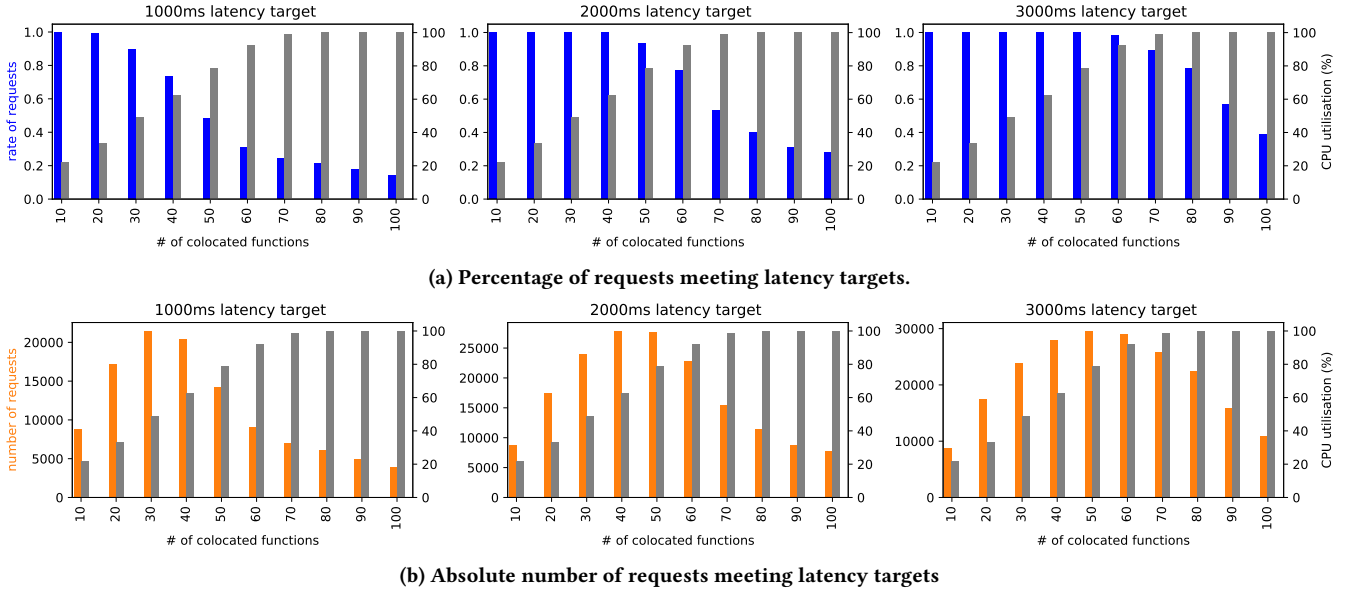


Figure 5: Impact of increasing contention scenarios in Figure 3 given 1000ms, 2000ms and 3000ms latency targets.

as larger targets of 2000ms and 3000ms to illustrate how the impact of contention varies in light of how strict latency targets are. Across all latency targets and for a smaller number of colocated functions, almost all requests meet their targets despite the skewed demand distribution. This demonstrates the advantage of the work-conserving principles of CFS, which enabled it to maximise the latency for all functions despite the equal CPU shares and the significant imbalance in demand of these functions. However, we start to see a gradual decrease in this percentage as we increase the number of colocated functions beyond a specific threshold.

We now demonstrate the impact of contention on the raw server performance which happens independently from the degree of CPU utilisation. Figure 5b illustrates the absolute number of requests which are executed successfully within the given latency targets. Across all latency targets, we observe that this number of requests increases as we increase the number of colocated functions until we reach a peak. Beyond this peak, we see a performance degradation where the server is incapable of delivering the same raw number of requests within the given latency target. For latency targets 1000ms and 2000ms, the performance degradation can occur even under moderate CPU utilisation and we observe more than 60% and 40% degradation respectively when CPU cores are fully utilised with 70 colocated functions.

**Conclusion 1:** Under low load, the work-conserving design principle enables CFS with the default settings to maximise the performance of colocated tasks even with a skewed demand distribution. However contention can occur as CPU resources get more utilised and detecting such contention is necessary to avoid performance degradation. This requires real-time monitoring of metrics beyond CPU utilisation.

Estimating the ideal number of colocated functions to avoid such degradation is highly dependent on the demand characteristics and latency requirements of these functions. For a latency target of 1000ms, we can have up to 20 colocated functions without a significant impact on latency, while this number can be increased

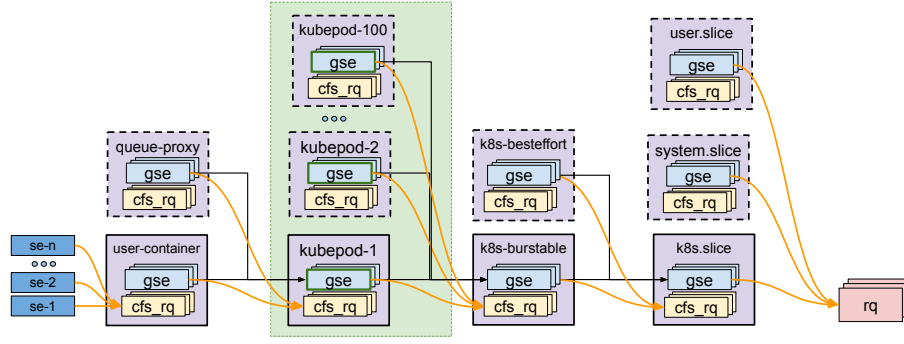
to 40 and 60 colocated functions for larger targets of 2000ms and 3000ms respectively. However, estimating such ideal limits a priori is difficult given the bursty and highly fluctuating demand patterns of serverless workloads.

**Conclusion 2:** Unexpected request bursts and over-estimation of node capacity in terms of the number of functions which can share the resources of a single cluster node is inevitable and requires intervention from the cluster scheduler to scale resources using other cluster nodes. The scheduling policy of the operating system plays a crucial role to mitigate the impact of this problem until proper action is taken by the cluster scheduler.

#### 4 Prototype of latency-aware CFS

In this section, we propose to adjust the CFS scheduler in order to mitigate the impact of contention on function latency under high CPU load in light of the unique characteristics of serverless workloads. The key assumption is that serverless workloads follow a skewed demand distribution [20, 24, 31, 38], similar to one discussed in §3.1 where a big chunk of demand is distributed among a large number of short-lived functions which are mostly idle. The key idea of the proposed CFS variant is to relax the fairness goal of CFS and replace that with a policy that gives priority to the long tail of least loaded functions. By dynamically prioritising these functions, we allow contended CPU run queues to drain more quickly, and other functions to execute with significantly less interruption. We achieve this change mainly by adjusting CFS's dynamic priority based on minimum virtual run time and leveraging an existing kernel mechanism that estimates the instantaneous load of tasks in the system which is known as per-entity load tracking (PELT) [4].

PELT mechanism enables to track the load on the granularity of every scheduling entity, and quantifies the nature of the task whether it is bursty or using the CPU more steadily. CFS aggregates this metric on the level of every cgroup via the corresponding task group data structure `tg->load_avg`. The key idea which makes



**Figure 6: Hierarchical task group data structure used by CFS to represent the cgroup tree in Figure 1. Each cgroup corresponds to a task group entity (tg) which is represented as a purple box along with its corresponding high-level scheduling entities (gse) and run queues (cfs\_rq). The inner dimension represents data structures which are replicated for each CPU core. Task groups with solid boxes have fully expanded child groups. Every scheduling entity waits on the run queue owned by its parent which is illustrated through the black and orange lines. Task groups highlighted in green correspond to function sandboxes which can be configured via the `cpu.func_sandbox` cgroup property we introduce in §4.**

our CFS patch extremely lightweight is the reuse of the load metric aggregated via the PELT mechanism on the level of cgroups that correspond to function sandboxes (highlighted in Figure 6). When a mostly idle function becomes runnable, it would have a low load value, and its tasks are given priority over tasks of functions with higher load values. We introduce a new cgroup property `cpu.func_sandbox` which can be configured based on the serverless framework to inform the CFS scheduler of cgroups that correspond to a function sandbox.

We realise our proposed CFS variant with a proof-of-concept patch in which we introduce two adjustments to the CFS scheduler code (`sched/fair.c`). The first is to internally and dynamically adjust the `cpu.shares` property of cgroups corresponding to the serverless functions to be inverse proportional to the load of their corresponding task groups. That is, a function that has the least load value will dynamically receive the largest timeshare. The second adjustment is to use the value of the timeshare itself as a new dynamic priority for CFS where tasks with the largest timeshare will be at the front of CFS run queues. In other words, we use the largest timeshare as a proxy for the least loaded function, which enables us to prototype a CFS variant with few lines of code. The proposed CFS variant internally overrides any static shares which might be configured by the Kubernetes via the `cpu.shares` cgroup interface. Nevertheless, Kubernetes will still be able to use these shares to bin pack [12] function pods across the cluster nodes.

## 5 Evaluation

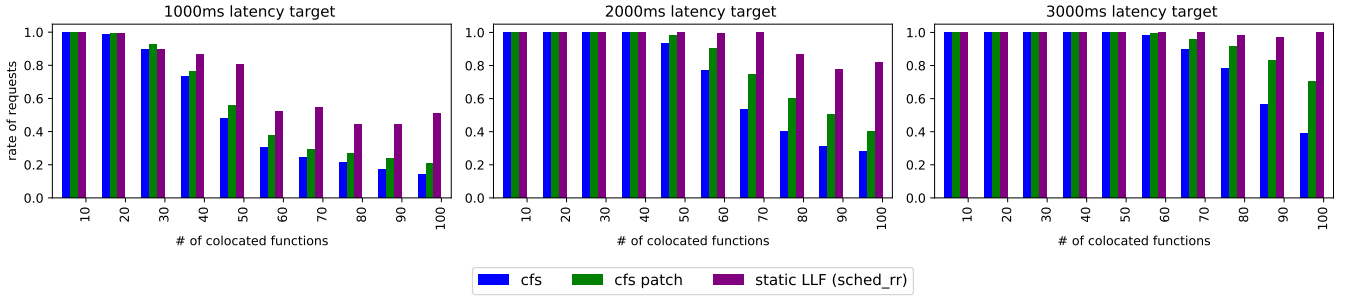
**Evaluation setup and workload generation.** We use a small-scale Kubernetes cluster composed of a number of dedicated servers based on Intel(R) Xeon(R) CPU E5-2430L with 6 physical cores (2 hardware threads per core) and 64 GB of memory. We use Kubernetes version 1.23 where cluster nodes are configured with recent stable kernel version 5.18 and containerd 1.4.12. We use the PyTorch framework for the image classification model which is wrapped by BentoML, an ML serving framework which generates an HTTP API around the model. The target workload is deployed on a single worker node that has the most necessary management

workloads such as the Kublet agent. We deploy 100 functions on the worker node in order to evaluate the contention scenarios presented in §3.1 which are made readily available before all experiments as we consider cold start an orthogonal concern. We use the trace segments extracted from the Azure Functions Invocation Trace [41] as described in §3.1 to derive precise per-function inter-arrival times incorporating diverse burst arrival patterns. Each trace segments is replayed by an open loop generator which sends high resolution frames of 720x1280 pixels to a corresponding image classification function over HTTP.

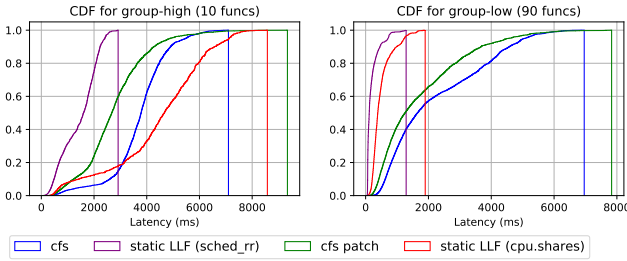
**Evaluation baselines.** We evaluate our proposed CFS variant (`cfs-patch`) and compare it to vanilla CFS (`cfs`) under the default settings of shares as configured by Kubernetes. We also consider two additional baselines. The first is a static approximation of a least loaded first scheduling policy which we achieve by statically prioritising a number of functions that belong to the lowest levels of demand via `SCHED_RR`,<sup>2</sup> a soft real-time scheduling policy, while the rest of functions remain under the default CFS policy. This results with a hybrid scheduling setup (static LLF (`sched_rr`)) in which we service low load tasks (**group-low**) more quickly and allocate any time left to the high load tasks (**group-high**). Figure 3b illustrates the two groups when this setup is applied to the worst contention scenario discussed in §3.1. We arrive at the ideal size of (**group-low**) experimentally by incrementally adding new functions to the group as long as contention within that group does not have visible impact on latency targets. We found that all functions of the lowest nine demand bands can be included in the low load group. We also replicate this static approximation of LLF solely based on CFS (static LLF (`cpu.shares`)) to demonstrate the necessity of relaxing fairness. We use the cgroup `cpu.shares` property to increase the timeshare of the low loaded group which is allocated the maximum share available in CFS.

**Latency CDFs.** Figure 8 illustrates the latency CDFs for the four scheduling setups describe above under the worst contention scenario (100 pods), where we compare the CDFs for the two load

<sup>2</sup>Tasks under `SCHED_RR` can preempt other CFS tasks immediately and by default can take up to 95% of a scheduling period of one second.



**Figure 7: Comparison of percentage of requests meeting latency targets under CFS baseline, patched CFS and a static approximation of LLF. This is under increasing contention scenarios in Figure 3 and reported in Figure 5**



**Figure 8: Latency CDFs under worst contention scenario. Functions are spitted into high load and load groups according to Figure 3b to compare patched CFS to a static approximation of LLF policy.**

groups (**group-low**) and (**group-high**) separately. Examining the CDFs for (**group-low**), static LLF (sched\_rr) leads to the best latency distribution due to the high preemption priority of this group under SCHED\_RR. The counter intuitive insight is that this also leads to a significant improvement for (**group-high**) compared to vanilla CFS, although this group remain under the same CFS policy in both setups. Allocating **group-low** under a soft real-time policy allows the system to service these tasks more quickly within the scheduling period dedicated for SCHED\_RR tasks. When the time comes for (**group-high**) tasks to execute within CFS, these tasks run with significantly less interruption. Our proposed CFS achieves this approximation dynamically and helps mitigate the impact of contention for both groups of functions. This is in contrast to static LLF (cpu.shares) where the improvement for (**group-low**) exacerbates the degradation for (**group-high**). This is because increasing the timeshare under round-robin scheduling will always increase the turnaround time for other tasks.

**Attainment of latency targets.** We also evaluate the impact of the proposed approach on attainment of latency targets given the contention scenarios and latency targets discussed in §3. In Figure 7, we compare our proposed approach to the baseline CFS setup as well as to static LLF (sched\_rr) which we use to demonstrate the potential margin for improvement. We notice that static LLF (sched\_rr) introduces an improvement which is less significant for the 1000ms latency target, due to the intrinsic server overload which needs to be addressed by external resources. However, for the 2000ms and 3000ms latency targets, we can have at least 10-20 more colocated functions before a degradation in

performance occurs. Our CFS prototype increases the percentage of requests which meet their latency target across all contention scenarios and all latency targets with an improvement that is more significant for larger latency targets. Under the worst contention scenario, our prototype introduces around 6%, 12% and 30% improvements for 1000ms, 2000ms and 3000ms latency targets respectively, in comparison to vanilla CFS.

## 6 Conclusions and discussion

In this paper, we showed the potential of operating system scheduling setups that prioritise the least loaded tasks for serverless-style bursty workloads. Given the large number of mostly idle functions which generate demand over short periods of time, prioritising these functions allows to reduce contention on other running functions and improves the overall latency distribution. We demonstrate the potential to realise this priority within Linux CFS by leveraging its load tracking mechanism known as PELT, which we use as a dynamic priority to replace the round-robin style of scheduling in CFS. We also demonstrate how this can seamlessly integrate with an open source serverless framework such as Knative. Furthermore, unlike vertical autoscaling solutions [13, 35], our approach provides a highly responsive local mechanism to delay or mitigate the impact of contention without coordination with the control plane of Kubernetes.

Our evaluation of the current CFS patch shows that a significant margin for improvement remains viable and we aim to further improve our prototype in order to provide more predictable performance for the long tail of least loaded functions. We envision a local-first serverless cluster resource management approach, in which the operating system scheduler prioritises the majority of mostly idle functions and leaves overloaded functions to be handled by overload mechanisms of the cluster scheduler. We believe such a policy makes sense from both perspectives of fairness and efficiency as it would allow to reserve data locality and other factors taken into placement decisions that were made for a large number of functions, while also mitigating the impact of contention on hotspot functions. To this end, we aim to explore how this in-kernel, local-first approach can be integrated with cluster request routing and horizontal autoscaling in order to provide highly responsive overload management capabilities for clustered serverless environments.

## References

- [1] 15 February 2013. Amazon EMR Serverless supports larger worker sizes to run more compute and memory-intensive workloads. <https://aws.amazon.com/blogs/big-data/amazon-emr-serverless-supports-larger-worker-sizes-to-run-more-compute-and-memory-intensive-workloads/>.
- [2] 2023. Knative. <https://knative.dev/>.
- [3] 8 February 2021. Messaging Fanout Pattern for Serverless Architectures Using Amazon SNS. <https://aws.amazon.com/blogs/compute/messaging-fanout-pattern-for-serverless-architectures-using-amazon-sns/>.
- [4] April 15, 2015. Load tracking in the scheduler. <https://lwn.net/Articles/639543/>.
- [5] August 24, 2022. For the love of god, stop using CPU limits on Kubernetes. <https://home.robusta.dev/blog/stop-using-cpu-limits>.
- [6] February 8, 2021. Comparing Serverless Performance for CPU Bound Tasks. <https://blog.cloudflare.com/serverless-performance-with-cpu-bound-tasks/>.
- [7] February 8, 2021. The burstable CFS bandwidth controller. <https://lwn.net/Articles/844976/>.
- [8] July 2, 2007. CFS group scheduling. <https://lwn.net/Articles/240474/>.
- [9] March 17, 2022. Control Group v2, The Linux Kernel Documentation. <https://docs.kernel.org/admin-guide/cgroup-v2.html>.
- [10] March 17, 2022. Improved response times with latency nice. <https://lwn.net/Articles/887842/>.
- [11] March 17, 2022. introduce cpu.headroom knob to cpu controller (Linux patch). <https://lore.kernel.org/lkml/20190408214539.2705660-1-songliubraving@fb.com/>.
- [12] March 17, 2022. Kubernetes Resource Bin Packing, Kubernetes Documentation. <https://kubernetes.io/docs/concepts/scheduling-eviction/resource-bin-packing/>.
- [13] March 17, 2022. Kubernetes Vertical Pod Autoscaler. <https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler>.
- [14] November 26, 2019. Fixing SCHED\_IDLE. <https://lwn.net/Articles/805317/>.
- [15] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*. USENIX Association, Santa Clara, CA, 419–434. <https://www.usenix.org/conference/nsdi20/presentation/agache>
- [16] M. Aslanpour, A. Toosi, C. Cicconetti, B. Javadi, Peter Sbarski, D. Taibi, M. Asunção, S. S. Gill, R. Gaire, and S. Dustdar. 2021. Serverless Edge Computing: Vision and Challenges. *2021 Australasian Computer Science Week Multiconference (2021)*.
- [17] Shuang Chen, Christina Delimitrou, and José F. Martínez. 2019. PARTIES: QoS-Aware Resource Partitioning for Multiple Interactive Services. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. Association for Computing Machinery, New York, NY, USA, 107–120. <https://doi.org/10.1145/3297858.3304005>
- [18] Simon Eismann, Joel Scheuner, Erwin Van Eyk, Maximilian Schwinger, Johannes Grohmann, Nikolas Roman Herbst, Cristina L. Abad, and Alexandru Iosup. 2020. A Review of Serverless Use Cases and their Characteristics. *ArXiv abs/2008.11110* (2020).
- [19] Yuqi Fu, Li Liu, Haoliang Wang, Yue Cheng, and Songqing Chen. 2022. SFS: smart OS scheduling for serverless functions. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC '22)*. IEEE Press, Dallas, Texas, 1–16.
- [20] Alexander Fuerst and Prateek Sharma. 2022. Locality-Aware Load-Balancing For Serverless Clusters. In *Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing (Minneapolis, MN, USA) (HPDC '22)*. Association for Computing Machinery, New York, NY, USA, 227–239. <https://doi.org/10.1145/3502181.3531459>
- [21] Arpan Gujarati, Sameh Elnikety, Yuxiong He, Kathryn S. McKinley, and Björn B. Brandenburg. 2017. Swayam: Distributed Autoscaling to Meet SLAs of Machine Learning Inference Services with Resource Efficiency. In *Proceedings of the 18th ACM/IFIP/USENIX Middleware Conference (Las Vegas, Nevada) (Middleware '17)*. Association for Computing Machinery, New York, NY, USA, 109–120. <https://doi.org/10.1145/3135974.3135993>
- [22] Weiwei Jia, Jiuyan Zhang, Jianchen Shan, Jing Li, and Xiaoning Ding. 2022. Achieving Low Latency in Public Edges by Hiding Workloads Mutual Interference. In *Proceedings of the 13th Symposium on Cloud Computing (San Francisco, California) (SoCC '22)*. Association for Computing Machinery, New York, NY, USA, 477–492. <https://doi.org/10.1145/3542929.3563459>
- [23] Kostis Kaffes, Neeraja Jayant Yadwadkar, and Christos Kozyrakis. 2021. Practical Scheduling for Real-World Serverless Computing. *ArXiv abs/2111.07226* (2021).
- [24] Kostis Kaffes, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2022. Hermod: principled and practical scheduling for serverless functions. In *Proceedings of the 13th Symposium on Cloud Computing*. ACM, San Francisco California, 289–305. <https://doi.org/10.1145/3542929.3563468>
- [25] Jeongchul Kim and Kyungyong Lee. 2019. FunctionBench: A Suite of Workloads for Serverless Cloud Function Service. *2019 IEEE 12th International Conference on Cloud Computing (CLOUD) (2019)*, 502–504.
- [26] Baptiste Lepers, Redha Gouicem, Damien Carver, Jean-Pierre Lozi, Nicolas Palix, Maria-Virginia Aponte, Willy Zwaenepoel, Julien Sopena, Julia Lawall, and Gilles Muller. 2020. Provable multicore schedulers with Ipanema: application to work conservation. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20)*. Association for Computing Machinery, New York, NY, USA, 1–16. <https://doi.org/10.1145/3342195.3387544>
- [27] Junfeng Li, Sameer G. Kulkarni, K. K. Ramakrishnan, and Dan Li. 2019. Understanding Open Source Serverless Platforms: Design Considerations and Performance. In *Proceedings of the 5th International Workshop on Serverless Computing (Davis, CA, USA) (WOSC '19)*. Association for Computing Machinery, New York, NY, USA, 37–42. <https://doi.org/10.1145/3366623.3368139>
- [28] Zijun Li, Jiagan Cheng, Quan Chen, Eryu Guan, Zizheng Bian, Yi Tao, Bin Zha, Qiang Wang, Weidong Han, and Minyi Guo. 2022. [RunD]: A Lightweight Secure Container Runtime for High-density Deployment and High-concurrency Startup in Serverless Computing. 53–68. <https://www.usenix.org/conference/atc22/presentation/li-zijun-rund>
- [29] David Lo, Liqun Cheng, Rama Govindaraju, Parthasarathy Ranganathan, and Christos Kozyrakis. 2015. Heracles: improving resource efficiency at scale. *ACM SIGARCH Computer Architecture News* 43, 3S (June 2015), 450–462. <https://doi.org/10.1145/2872887.2749475>
- [30] Jean-Pierre Lozi, Baptiste Lepers, Justin Funston, Fabien Gaud, Vivien Quéma, and Alexandra Fedorova. 2016. The Linux scheduler: a decade of wasted cores. In *Proceedings of the Eleventh European Conference on Computer Systems (EuroSys '16)*. Association for Computing Machinery, New York, NY, USA, 1–16. <https://doi.org/10.1145/2901318.2901326>
- [31] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. 2022. ORION and the Three Rights: Sizing, Bundling, and Prewarming for Serverless DAGs. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association, Carlsbad, CA, 303–320. <https://www.usenix.org/conference/osdi22/presentation/mahgoub>
- [32] Viyom Mittal, Shixiong Qi, Ratnadeep Bhattacharya, Xiaosu Lyu, Junfeng Li, Sameer G. Kulkarni, Dan Li, Jinho Hwang, K. K. Ramakrishnan, and Timothy Wood. 2021. Mu: An Efficient, Fair and Responsive Serverless Framework for Resource-Constrained Edge Clouds. In *Proceedings of the ACM Symposium on Cloud Computing (Seattle, WA, USA) (SoCC '21)*. Association for Computing Machinery, New York, NY, USA, 168–181. <https://doi.org/10.1145/3472883.3487014>
- [33] Djib Mvondo, Antonio Barbalace, Alain Tchana, and Gilles Muller. 2021. Tell Me When You Are Sleepy and What May Wake You Up!. In *Proceedings of the ACM Symposium on Cloud Computing (Seattle, WA, USA) (SoCC '21)*. Association for Computing Machinery, New York, NY, USA, 562–569. <https://doi.org/10.1145/3472883.3487013>
- [34] Hai Nguyen, Chaojie Zhang, Zhujun Xiao, and A. Chien. 2019. Real-time Serverless: Enabling Application Performance Guarantees. *Proceedings of the 5th International Workshop on Serverless Computing* (2019).
- [35] Krzysztof Rzdca, Paweł Findeisen, Jacek Swiderski, Przemysław Zych, Przemysław Broniek, Jarek Kusmerek, Paweł Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. 2020. Autopilot: Workload Autoscaling at Google. In *Proceedings of the Fifteenth European Conference on Computer Systems (Heraklion, Greece) (EuroSys '20)*. Association for Computing Machinery, New York, NY, USA, Article 16, 16 pages. <https://doi.org/10.1145/3342195.3387524>
- [36] Arjun Singhvi, Arjun Balasubramanian, Kevin Houck, Mohammed Danish Shaikh, Shivaram Venkataraman, and Aditya Akella. 2021. Atoll: A Scalable Low-Latency Serverless Platform. In *Proceedings of the ACM Symposium on Cloud Computing (Seattle, WA, USA) (SoCC '21)*. Association for Computing Machinery, New York, NY, USA, 138–152. <https://doi.org/10.1145/3472883.3486981>
- [37] R. Smith. 2007. An Overview of the Tesseract OCR Engine. In *Proceedings of the Ninth International Conference on Document Analysis and Recognition - Volume 02 (ICDAR '07)*. IEEE Computer Society, USA, 629–633.
- [38] Huangshi Tian, Suyi Li, Ao Wang, Wei Wang, Tianlong Wu, and Haoran Yang. 2022. Owl: Performance-Aware Scheduling for Resource-Efficient Function-as-a-Service Cloud. In *Proceedings of the 13th Symposium on Cloud Computing (San Francisco, California) (SoCC '22)*. Association for Computing Machinery, New York, NY, USA, 78–93. <https://doi.org/10.1145/3542929.3563470>
- [39] Bin Wang, A. Ali-Eldin, and P. Shenoy. 2021. LaSS: Running Latency Sensitive Serverless Computations at the Edge. *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing* (2021).
- [40] Yanan Yang, Laiping Zhao, Yiming Li, Huanyu Zhang, Jie Li, Mingyang Zhao, Xingzhen Chen, and Keqiu Li. 2022. INFless: a native serverless system for low-latency, high-throughput inference. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 768–781. <https://doi.org/10.1145/3503222.3507709>

- [41] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and Cheaper Serverless Computing on Harvested Resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) (*SOSP '21*). Association for Computing Machinery, New York, NY, USA, 724–739. <https://doi.org/10.1145/3477132.3483580>
- [42] Laiping Zhao, Yanan Yang, Yiming Li, Xian Zhou, and Keqiu Li. 2021. Understanding, predicting and scheduling serverless workloads under partial interference. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, St. Louis Missouri, 1–15. <https://doi.org/10.1145/3458817.3476215>